2026-6-29 Linux开发工具--包管理器

- 1. Linux 软件安装与包管理器
  - 1.1 Linux 安装软件的三种思路
    - 1.1.1 源代码安装
    - 1.1.2 使用发行版软件包
    - 1.1.3 使用包管理器
  - 1.2 软件包与包管理器的关系
    - 1.2.1 软件包像“APP”
    - 1.2.2 包管理器的核心工作
    - 1.2.3 不同发行版的包管理器
  - 1.3 Linux 软件生态
    - 1.3.1 怎样评价一款操作系统
    - 1.3.2 圈子是手段，生态才是结果
    - 1.3.3 开源、用户与商业价值
  - 1.4 一次软件安装背后的完整流程
    - 1.4.1 本地包管理器与远程软件源
    - 1.4.2 包管理器为什么知道去哪里下载
- 2. 软件源、镜像源与软件生态
  - 2.1 软件源是什么
    - 2.1.1 软件仓库
    - 2.1.2 软件源为什么需要配置
  - 2.2 什么是镜像源
    - 2.2.1 镜像的基本思路
    - 2.2.2 为什么换源能加速
    - 2.2.3 Ubuntu 换源流程
    - 2.2.4 CentOS yum 源思路
  - 2.3 官方源与第三方扩展源
    - 2.3.1 官方源的特点
    - 2.3.2 什么时候需要第三方源
    - 2.3.3 以 Docker 为例添加 apt 源
  - 2.4 软件包版本、架构与来源
    - 2.4.1 RPM 包名中的信息
- 3. 软件包依赖
  - 3.1 什么是依赖
    - 3.1.1 从 C 语言库理解
    - 3.1.2 依赖不是“可选项”

        - 3.1.3 自动依赖解析
        - 3.1.4 为什么会出现依赖冲突
    - 3.2 软件包信息如何帮助判断
        - 3.2.1 名称、版本、来源和依赖
- 4. yum 与 apt 常用操作
    - 4.1 两个包管理器的命令对照
        - 4.1.1 基本操作表
        - 4.1.2 apt 与 apt-get
    - 4.2 搜索与查看软件包
        - 4.2.1 搜索
        - 4.2.2 查看详细信息
    - 4.3 安装、卸载与升级
        - 4.3.1 安装
        - 4.3.2 卸载
        - 4.3.3 更新索引与升级软件
    - 4.4 缓存与清理
        - 4.4.1 为什么会有缓存
        - 4.4.2 常见清理命令
    - 4.5 软件包安装时的系统行为
        - 4.5.1 需要管理员权限
        - 4.5.2 一次完成一个安装任务
- 5. 下载安装软件示例：lrzsz
    - 5.1 搜索软件包
        - 5.1.1 搜索结果
        - 5.1.2 查看详情
    - 5.2 安装软件
    - 5.3 删除软件
- 6. 网络、联网测试与常见故障
    - 6.1 包管理器为什么依赖网络
        - 6.1.1 下载软件包索引
        - 6.1.2 下载软件包和依赖
    - 6.2 ping 能说明什么
        - 6.2.1 ping 可以测试什么
        - 6.2.2 ping 不能证明软件源可用
    - 6.3 常见问题排查
        - 6.3.1 apt update 报错
        - 6.3.2 yum 找不到包

        - 6.3.3 包管理器被锁住
        - 6.3.4 安装成功后却找不到命令
        - 6.3.5 磁盘空间不足
- 7. 包管理器操作流程与命令速查
    - 7.1 推荐安装流程
        - 7.1.1 操作前
        - 7.1.2 操作中
        - 7.1.3 操作后
    - 7.2 apt 速查
    - 7.3 yum 速查
    - 7.4 软件源速查
    - 7.5 本文核心主线

1. Linux 软件安装与包管理器

1.1 Linux 安装软件的三种思路

1.1.1 源代码安装

最原始的方式是取得软件源代码，在自己的机器上配置、编译，再把生成的可执行文件或库放到指定目录。
这种方式的优点是控制程度高，可以按自己的环境调整编译选项；缺点是步骤多，需要自行准备编译器和依赖，软件升级、卸载和兼容性维护也更麻烦。
一个简单流程可以理解为：

```
下载源代码 → 配置编译环境 → 编译 → 安装到指定目录
```

1.1.2 使用发行版软件包

为了避免每个用户都重复编译，发行版或第三方会提前把软件编译好，并连同版本、依赖、安装脚本等元数据一起打包。
常见形式包括：

- Debian/Ubuntu 的 `.deb` 软件包。
- Red Hat/CentOS/Fedora 的 `.rpm` 软件包。
  软件包不是单纯的压缩文件。它除了程序本身，还记录了版本、架构、依赖、安装路径、维护者等信息。

1.1.3 使用包管理器

包管理器的价值不只是“下载一个文件”，而是负责：

```
查找软件
→ 读取软件源元数据
→ 自动计算依赖
→ 下载软件包及依赖
→ 安装、升级或卸载
→ 维护系统中的软件状态
```

因此日常开发中，包管理器安装是效率最高、最容易维护的方式。

1.2 软件包与包管理器的关系

1.2.1 软件包像“APP”

软件包可以类比成手机上的应用：

- 软件包本身是已经整理好的程序。
- 包管理器负责搜索、获取、安装和卸载。
- 软件源则类似应用商店或软件仓库。

所以可以用一个直观的类比理解：

```
软件包        ≈ APP
包管理器      ≈ 应用商店 App
软件源        ≈ 应用商店的服务器
```

1.2.2 包管理器的核心工作

包管理器通常内置或读取软件源配置，从服务器获取软件包索引。它知道：

- 当前系统是什么发行版。
- 系统使用什么 CPU 架构。
- 本地已经安装了哪些软件。
- 某个软件需要哪些依赖。

- 哪些软件包可以升级。

用户只需要输入包名和操作，复杂的依赖与下载流程由包管理器处理。

1.2.3 不同发行版的包管理器

常见对应关系：

发行版家族	包格式	常用包管理器
Fedora、Red Hat、CentOS	RPM	`yum`，较新系统常用 `dnf`
Debian、Ubuntu	DEB	`apt`

课堂中主要比较：

```
CentOS → yum
Ubuntu → apt
```

`yum` 的全称通常写作 Yellowdog Updater, Modified；`apt` 的全称是 Advanced Package Tool。

1.3 Linux 软件生态

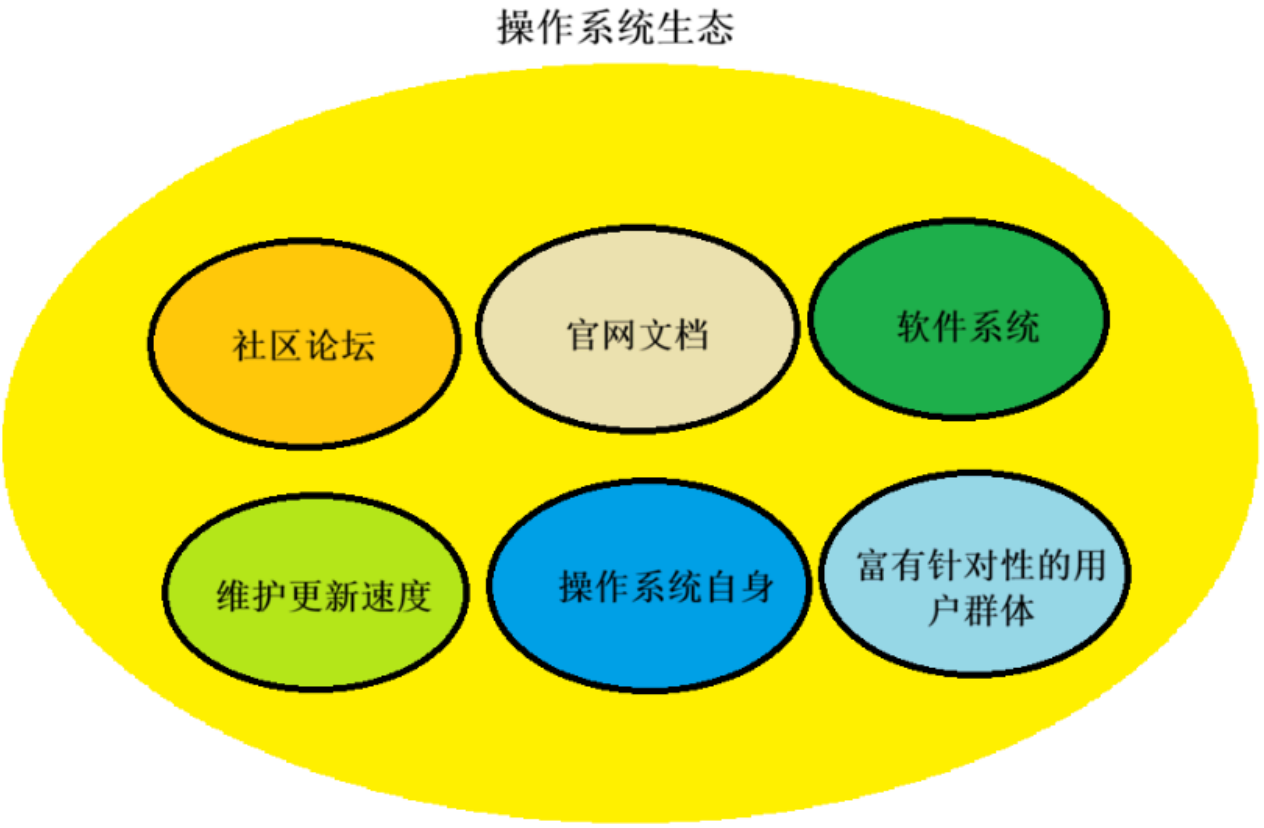
1.3.1 怎样评价一款操作系统

评价一个操作系统，不能只看内核本身，还要看它能否被持续使用和维护。

一个成熟的系统通常会形成：

- 用户社区。
- 配套文档与教程。
- 软件仓库。
- 第三方开发者。
- 企业支持与服务。
- 持续的更新与维护。

系统被更多人使用后，会吸引更多人开发软件、编写文档和参与维护；生态丰富后，又会进一步降低新用户的使用门槛。



1.3.2 圈子是手段，生态才是结果

社区、论坛和文档的形成不是最终目的。它们的作用是让系统更容易被使用，让用户、开发者和企业之间形成持续的价值交换。

一旦这种关系稳定下来，就会形成良性循环：

```
更多人使用
→ 更多软件与文档
→ 更好用
→ 更多人使用
```

1.3.3 开源、用户与商业价值

开源软件可以免费使用、修改和分发，但这不代表它没有维护成本。

一个生态中的参与者可能通过以下方式获得价值：

- 用户获得软件与服务。
- 开发者获得用户反馈、技术影响力和就业机会。
- 企业提供服务器、资金、云服务或商业支持。
- 企业基于开源项目提供发行版、托管和咨询服务。

从某种角度看，开源也可以成为一种商业模式。它依靠开放代码吸引用户和开发者，再通过服务、支持和生态影响力形成持续价值。

1.4 一次软件安装背后的完整流程

1.4.1 本地包管理器与远程软件源

可以先把整体结构理解成：

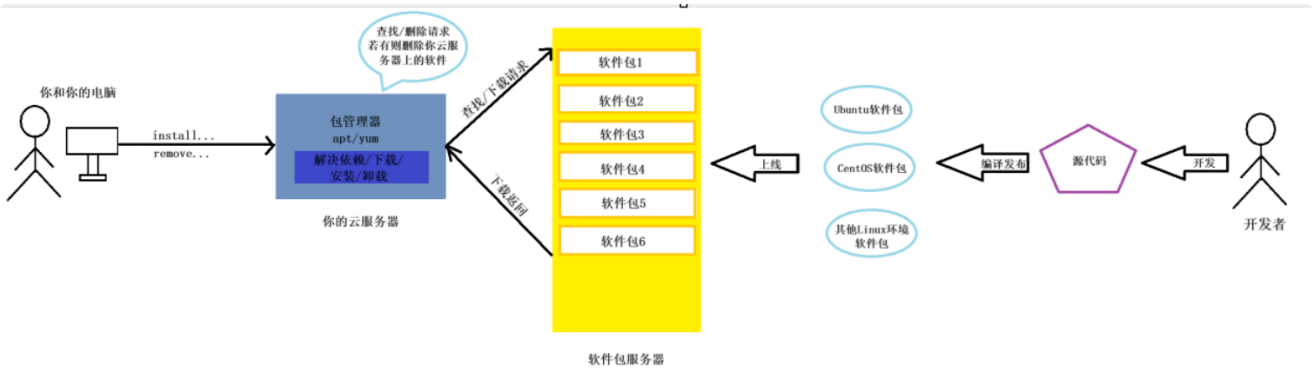
```
你的云服务器
    │
    └─ 包管理器 yum / apt
          ├─ 读取软件源配置
          ├─ 查询软件包索引
          ├─ 计算依赖
          └─ 发送下载请求

远程软件源服务器
    ├─ 软件包 1
    ├─ 软件包 2
    ├─ 软件包 3
    └─ ...
```

用户看到的可能只是：

```
sudo apt install 包名
```

但包管理器在后台完成搜索、依赖分析和下载。



1.4.2 包管理器为什么知道去哪里下载

下载软件时，包管理器必须先知道软件源服务器的地址。

系统中通常存在类似配置：

```
CentOS:
/etc/yum.repos.d/

Ubuntu:
/etc/apt/sources.list
/etc/apt/sources.list.d/
```

这些配置文件记录了软件仓库的 URL、发行版代号、组件和签名信息。

工作过程可以概括为：

```
读取配置
→ 得到仓库 URL
→ 下载或更新软件包索引
→ 根据本地状态和依赖关系计算安装方案
→ 下载并安装
```

2. 软件源、镜像源与软件生态

2.1 软件源是什么

2.1.1 软件仓库

软件源是一组可以被包管理器访问的软件仓库，其中包含：

- 软件包文件。
- 软件包索引。
- 版本和架构信息。
- 依赖关系。
- 仓库签名和校验信息。

它不只是“某个下载链接”，而是一整套能够被包管理器解析的软件元数据。

2.1.2 软件源为什么需要配置

如果没有软件源配置，包管理器不知道：

- 去哪里查询软件包。
- 当前系统应该使用哪个发行版仓库。
- 使用哪种 CPU 架构。
- 如何验证软件包没有被篡改。

所以安装软件前，经常需要先确认软件源配置并更新索引。

2.2 什么是镜像源

2.2.1 镜像的基本思路

许多 Linux 发行版及其官方软件仓库最初部署在国外。国内用户访问国外服务器时，可能因为物理距离、网络链路或其他限制而速度较慢、连接不稳定。

镜像站会把官方仓库中的软件包和索引同步到国内服务器。用户把软件源地址切换到镜像站后，就可以通过更近的网络节点下载。

这个过程可以理解为：

```
国外官方仓库
      ↓ 同步
国内镜像站
      ↓ 下载
本地包管理器
```

镜像同步的是软件和索引的副本，通常不会改变软件本身的功能。

2.2.2 为什么换源能加速

更换软件源的实质是把配置中的仓库地址替换成可访问的镜像地址。

但只更换网址还不够，必须满足以下条件：

- 镜像站仍然可用。
- 镜像覆盖当前发行版版本。
- 发行版代号匹配。
- CPU 架构匹配。
- 仓库签名和 GPG 配置正确。

否则可能出现：

```
404 Not Found
Release file is not valid
NO_PUBKEY
```

2.2.3 Ubuntu 换源流程

先查看当前配置：

```
cat /etc/apt/sources.list
ll /etc/apt/sources.list.d/
```

```
ZL@VM-0-14-ubuntu:~/code/lesson1$ cat /etc/apt/sources.list.d/ubuntu.sources
Types: deb
#URIs: http://archive.ubuntu.com/ubuntu/
URIs: http://mirrors.tencentyun.com/ubuntu/
Suites: noble noble-updates noble-backports
Components: main restricted universe multiverse
Signed-By: /usr/share/keyrings/ubuntu-archive-keyring.gpg

Types: deb
#URIs: http://security.ubuntu.com/ubuntu/
URIs: http://mirrors.tencentyun.com/ubuntu/
Suites: noble-security
Components: main restricted universe multiverse
Signed-By: /usr/share/keyrings/ubuntu-archive-keyring.gpg
ZL@VM-0-14-ubuntu:~/code/lesson1$
```

备份原配置：

```
sudo cp /etc/apt/sources.list /etc/apt/sources.list.bak
```

修改主配置文件：

```
sudo vim /etc/apt/sources.list
```

注意：`/etc/apt/sources.list.d/` 是目录，不是单个配置文件。需要修改其中某个 `.list` 或 `.sources` 文件时，应指定具体文件。



更新索引并清理下载缓存：

```
sudo apt update
sudo apt clean
sudo apt autoclean
```

三者的区别：

命令	作用
`apt update`	从软件源更新软件包索引，不会升级已安装软件
`apt clean`	清除已下载的软件包缓存
`apt autoclean`	只清理已经过期或不再需要的包缓存

需要真正升级软件：

```
sudo apt upgrade
```

2.2.4 CentOS yum 源思路

CentOS 一般通过 `/etc/yum.repos.d/` 中的 `.repo` 文件配置仓库。

查看仓库：

```
ll /etc/yum.repos.d/
```

标准的 CentOS 仓库文件可能是：

```
/etc/yum.repos.d/CentOS-Base.repo
```

扩展仓库文件可能是：

```
/etc/yum.repos.d/epel.repo
```

课堂中安装扩展源：

```
sudo yum install -y epel-release
```

更换仓库文件后，清理旧索引并重新生成缓存：

```
sudo yum clean all
sudo yum makecache
```

查看仓库列表：

```
sudo yum repolist
```

CentOS 7 已经进入停止常规维护阶段，某些旧镜像地址可能失效。实际换源时必须先确认仓库仍然可用，不要照搬过期链接。

2.3 官方源与第三方扩展源

2.3.1 官方源的特点

发行版官方仓库通常优先收录：

- 稳定版本软件。
- 开源软件。
- 经过维护者审核的软件。
- 与当前发行版兼容的依赖版本。

因此官方源不一定包含某个软件的最新版本，也可能不包含闭源软件或特定厂商驱动。

2.3.2 什么时候需要第三方源

以下情况可能需要添加第三方软件源：

- 软件版本更新很快，官方源太旧。
- 软件属于特定厂商维护。
- 软件不在发行版官方仓库中。

典型例子包括：

- Docker。
- Node.js 新版。
- Nginx 特定版本。
- NVIDIA 驱动。
- VS Code。

第三方源提高了软件选择范围，但也意味着系统要信任额外的维护者。

2.3.3 以 Docker 为例添加 apt 源

第一步，添加 Docker 仓库签名密钥：

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
```

第二步，创建 Docker 仓库配置：

```
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/docker-archive-
keyring.gpg] \
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

第三步，让 apt 读取新仓库：

```
sudo apt update
```

第四步，从新仓库安装：

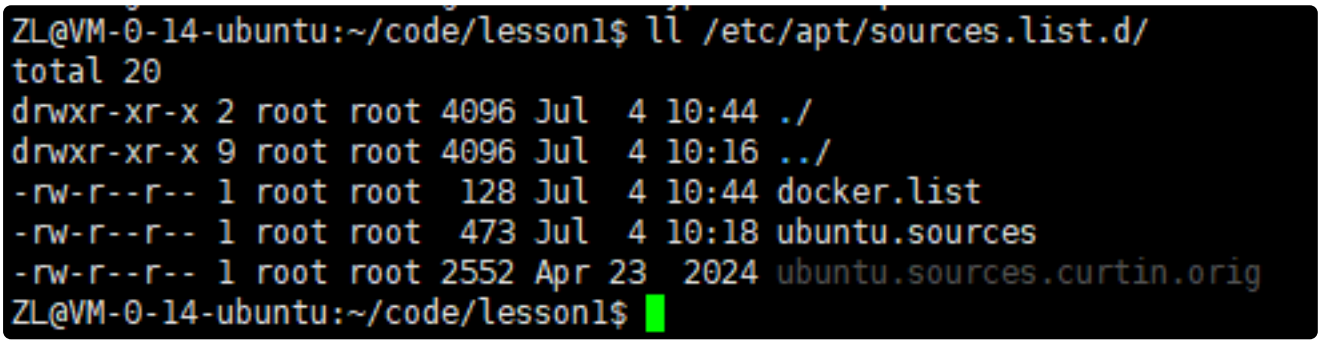
```
sudo apt install docker-ce
```

这里的关键点：

1. `gpg` 密钥用于验证软件包来源。
2. `signed-by=` 指定信任哪个密钥。
3. 新增仓库后必须先 `apt update`。
4. CPU 架构必须与仓库配置匹配。

查看已经添加的第三方源：

```
ll /etc/apt/sources.list.d/
```



2.4 软件包版本、架构与来源

2.4.1 RPM 包名中的信息

CentOS 系列中，包名常带有架构和发行版信息，例如：

```
lrzsz.x86_64 0.12.20-36.el7 @base
```

可以拆成：

部分	含义
`lrzsz`	软件包名称
`0.12.20`	软件版本
`36`	软件包的发行的修订号
`el7`	面向 RHEL/CentOS 7 系列
`x86_64`	64 位 x86 架构
`@base`	来自名称为 base 的软件源

常见架构后缀：

- `x86_64`：64 位 x86 系统。
- `i686`：32 位 x86 系统。
- `aarch64`：64 位 ARM 架构。

选择软件包时，包名和系统架构必须匹配。

3. 软件包依赖

3.1 什么是依赖

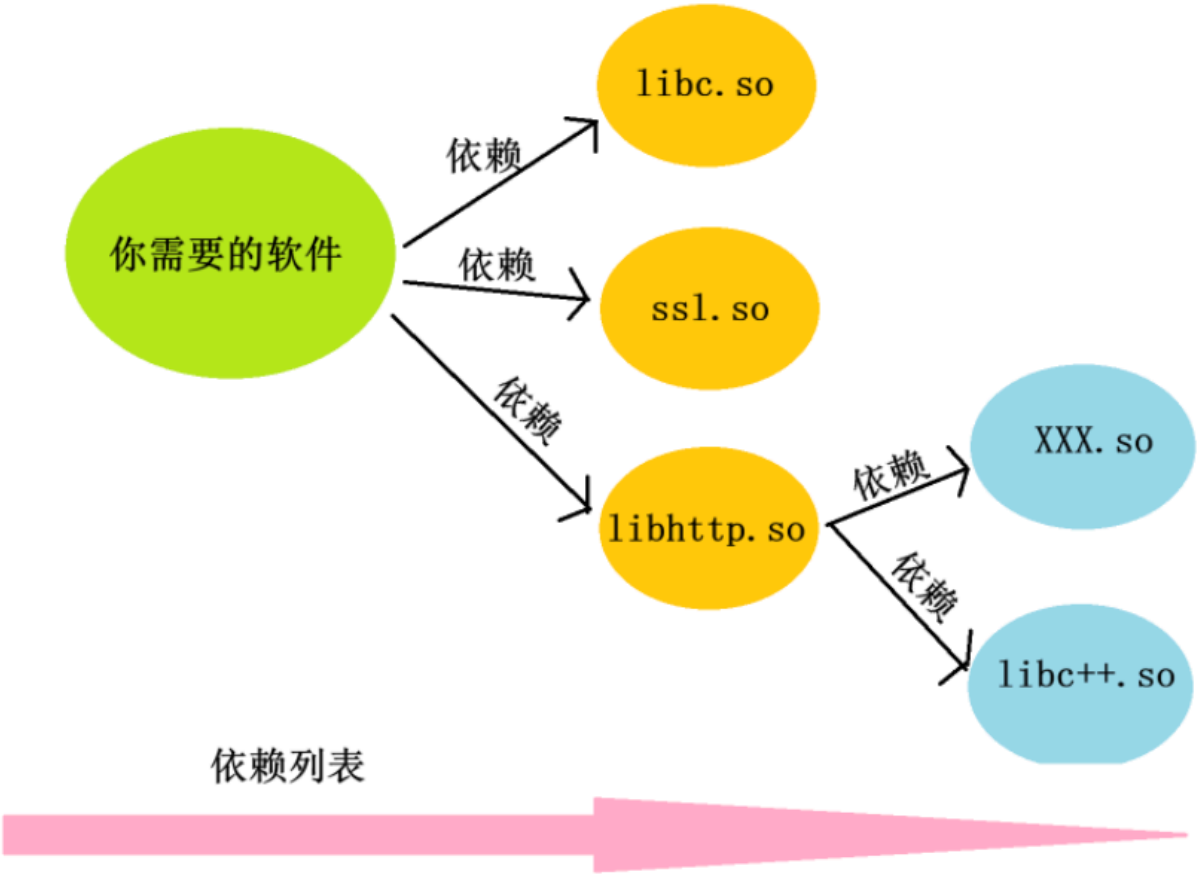
3.1.1 从 C 语言库理解

写 C 程序时，代码可能调用标准库或其他库中的函数。程序自身没有实现这些函数，而是依赖库提供实现。

软件包也是同样的道理：

```
软件 A
  ├─ 依赖运行库 B
  ├─ 依赖配置工具 C
  └─ 依赖组件 D
```

包管理器会读取软件包的 `Depends` 等字段，自动找到缺失的依赖并一起安装。



3.1.2 依赖不是“可选项”

需要区分几种关系：

元数据	含义
`Depends`	运行或安装所必需的依赖
`Recommends`	推荐安装，缺少时通常仍可运行
`Suggests`	建议安装，属于额外增强功能
`Conflicts`	不能同时安装或使用的包

缺少必需依赖时，安装通常会失败；缺少 `Suggests` 中的包不一定影响核心功能。

3.1.3 自动依赖解析

用户通常只需要安装目标软件：

```
sudo apt install docker-ce
```

包管理器会自动计算：

```
docker-ce
→ 需要哪些库
→ 这些库又需要哪些包
→ 哪些已经安装
→ 还需要下载哪些
```

依赖关系可能有很深的层级，手工处理非常容易遗漏。

3.1.4 为什么会出现依赖冲突

不同软件可能要求同一个库的不同版本，或者依赖关系的闭环发生变化，因此可能出现冲突。

常见现象：

- 需要安装较低版本的依赖。
- 两个软件不能同时安装。
- 某个包破坏了系统中的其他包。
- 第三方源与官方源的版本不兼容。

遇到冲突时应先阅读包管理器给出的具体信息，不要盲目执行强制参数。

3.2 软件包信息如何帮助判断

3.2.1 名称、版本、来源和依赖

查看包信息时，通常会看到：

```
Package         软件包名称
Version         版本
Priority        优先级
Section         分类区域
Origin          来源发行版
Maintainer      当前维护者
Depends         必需依赖
Suggests        建议包
Installed-Size  安装后大小
Download-Size   下载大小
APT-Sources     软件源与架构
Description     软件描述
```

这些字段能帮助判断：

- 是否是目标软件。
- 是否需要额外依赖。
- 下载和安装会占用多少空间。
- 软件来自官方源还是第三方源。
- 当前版本是否适合系统。

4. yum 与 apt 常用操作

4.1 两个包管理器的命令对照

4.1.1 基本操作表

目的	yum / CentOS	apt / Ubuntu
更新软件包索引	`sudo yum makecache`	`sudo apt update`
升级已安装软件	`sudo yum update`	`sudo apt upgrade`
搜索	`yum search 关键词`	`apt search 关键词`
列出包	`yum list | grep 关键词`	`apt list | grep 关键词`
查看详情	`yum info 包名`	`apt show 包名`
安装	`sudo yum install -y 包名`	`sudo apt install -y 包名`
卸载	`sudo yum remove -y 包名`	`sudo apt remove -y 包名`
清理缓存	`sudo yum clean all`	`sudo apt clean`
查看仓库	`yum repolist`	`apt policy`

4.1.2 apt 与 apt-get

`apt` 是面向普通用户的命令入口，输出更友好，适合交互式操作；`apt-get` 更偏向兼容旧脚本，很多自动化脚本仍在使用。

日常学习可以以 `apt` 为主，但看到 `apt-get` 时要知道它处理的是同一类软件包操作。

4.2 搜索与查看软件包

4.2.1 搜索

Ubuntu：

```
apt search lrzsz
```

CentOS：

```
yum search lrzsz
```

也可以列出包并过滤：

```
yum list | grep lrzsz
apt list | grep lrzsz
```

4.2.2 查看详细信息

```
apt show lrzsz
yum info lrzsz
```

查看已安装的包：

```
apt list --installed | grep lrzsz
dpkg -l | grep lrzsz
rpm -qa | grep lrzsz
```

4.3 安装、卸载与升级

4.3.1 安装

```
sudo apt install -y lrzsz
sudo yum install -y lrzsz
```

`-y` 表示自动确认安装提示。

安装前建议先查看包信息，确认包名、来源和版本。不要在不清楚包名的情况下直接复制陌生安装命令。

4.3.2 卸载

```
sudo apt remove -y lrzsz
sudo yum remove -y lrzsz
```

不同包管理器的“卸载”细节不完全相同：

- `apt remove` 通常保留系统级配置文件。
- `apt purge` 通常会连同配置文件一起删除。
- 需要谨慎使用会删除更多内容的清理命令。

4.3.3 更新索引与升级软件

Ubuntu 中：

```
sudo apt update
sudo apt upgrade
```

两者的区别非常重要：

```
apt update   → 更新“可安装/可升级”的索引
apt upgrade  → 根据索引真正升级软件包
```

CentOS 中可以用：

```
sudo yum check-update
sudo yum update
```

生产环境升级前应先检查变更内容、备份重要配置，并在测试环境验证。

4.4 缓存与清理

4.4.1 为什么会有缓存

包管理器下载软件包后，可能把安装包保留在本地缓存目录，便于重装或恢复。

长期使用后，缓存可能占用大量磁盘空间。

4.4.2 常见清理命令

```
sudo apt clean
sudo apt autoclean
sudo yum clean all
```

区别：

- `apt clean` 清理全部已下载包缓存。
- `apt autoclean` 只清理已过期、通常不再需要的缓存。
- `yum clean all` 清理 yum 缓存的元数据和包文件。

清理缓存不会卸载已经安装的软件。

4.5 软件包安装时的系统行为

4.5.1 需要管理员权限

安装软件通常会写入系统目录，例如：

```
/usr/bin
/usr/lib
```

```
/var/lib
/etc
```

因此通常需要使用：

```
sudo apt install 包名
sudo yum install 包名
```

这里的重点是系统目录写入权限，而不是包管理器本身的语法。

4.5.2 一次完成一个安装任务

apt/dpkg 或 yum 正在修改软件数据库时，会持有锁。此时再启动另一个安装命令通常会失败。

正确做法：

1. 等待当前安装结束。
2. 检查是否还有包管理进程在运行。
3. 重新执行操作。

不要把删除锁文件当成默认解决方案，否则可能破坏软件数据库一致性。

5. 下载安装软件示例：lrzsz

5.1 搜索软件包

5.1.1 搜索结果

以 Ubuntu 为例：

```
apt search lrzsz
```

```
ZL@VM-0-14-ubuntu:~/code/lesson1$ apt search lrzsz
Sorting... Done
Full Text Search... Done
cutecom/noble 0.51.0-1build2 amd64
  Graphical serial terminal, like minicom

lrzsz/noble,now 0.12.21-11build1 amd64 [installed]
  Tools for zmodem/xmodem/ymodem file transfer

ZL@VM-0-14-ubuntu:~/code/lesson1$
```

搜索结果通常会显示：

- 包名。
- 版本。
- 架构。
- 简短描述。
- 是否已经安装。

5.1.2 查看详情

```
apt show lrzsz
```

```
ZL@VM-0-14-ubuntu:~/code/lesson1$ apt show lrzsz
Package: lrzsz
Version: 0.12.21-11build1
Priority: optional
Section: universe/comm
Origin: Ubuntu
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Original-Maintainer: Martin A. Godisch <godisch@debian.org>
Bugs: https://bugs.launchpad.net/ubuntu/+filebug
Installed-Size: 237 kB
Depends: libc6 (>= 2.34)
Suggests: minicom
Homepage: https://ohse.de/uwe/software/lrzsz.html
Download-Size: 78.1 kB
APT-Manual-Installed: yes
APT-Sources: http://mirrors.tencentyun.com/ubuntu noble/universe amd64 Packages
Description: Tools for zmodem/xmodem/ymodem file transfer
 Lrzsz is a cosmetically modified zmodem/ymodem/xmodem package built
 from the public-domain version of Chuck Forsberg's rzsz package.
 .
 These programs use error correcting protocols ({z,x,y}modem) to send
 (sz, sx, sb) and receive (rz, rx, rb) files over a dial-in serial port
 from a variety of programs running under various operating systems.
```

课堂中看到的典型信息包括：

字段	含义
`Package: lrzsz`	软件包名称
`Version: 0.12.21-11build1`	软件版本
`Priority: optional`	可选软件，不是系统必须
`Section: universe/comm`	社区维护的通信相关工具
`Origin: Ubuntu`	来自 Ubuntu 发行环境
`Maintainer`	当前维护者
`Original-Maintainer`	原始维护者
`Bugs`	问题反馈入口
`Installed-Size`	安装后占用空间
`Depends: libc6 (>= 2.34)`	依赖 C 标准库，且版本不能过低
`Suggests: minicom`	建议搭配的工具，不是硬性依赖
`Homepage`	软件官网
`Download-Size`	下载大小
`APT-Manual-Installed`	是否为用户手动安装
`APT-Sources`	实际软件源、发行版和架构
`Description`	软件功能说明

`lrzsz` 提供 ZMODEM、XMODEM、YMODEM 相关文件传输工具，常用于串口或终端环境。

5.2 安装软件

```
sudo apt install -y lrzsz
```

```
ZL@VM-0-14-ubuntu:~/code/lesson1$ sudo apt install -y lrzsz
[sudo] password for ZL:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
lrzsz is already the newest version (0.12.21-11build1).
0 upgraded, 0 newly installed, 0 to remove and 128 not upgraded.
ZL@VM-0-14-ubuntu:~/code/lesson1$
```

安装过程中，包管理器会自动处理依赖。如果依赖缺失，可能先下载并安装依赖，再安装目标软件。

5.3 删除软件

```
sudo apt remove -y lrzsz
```

```
ZL@VM-0-14-ubuntu:~/code/lesson1$ sudo apt remove -y lrzsz
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages will be REMOVED:
  lrzsz
0 upgraded, 0 newly installed, 1 to remove and 128 not upgraded.
After this operation, 237 kB disk space will be freed.
(Reading database ... 89851 files and directories currently installed.)
Removing lrzsz (0.12.21-11build1) ...
Processing triggers for man-db (2.12.0-4build2) ...
ZL@VM-0-14-ubuntu:~/code/lesson1$
```

如果还要清除配置文件，可了解：

```
sudo apt purge lrzsz
```

执行 purge 前要确认是否还需要保留配置。

6. 网络、联网测试与常见故障

6.1 包管理器为什么依赖网络

6.1.1 下载软件包索引

`apt update`、`yum makecache` 都需要访问软件源服务器。没有网络、DNS 无法解析或仓库不可达时，更新索引就会失败。

6.1.2 下载软件包和依赖

即使索引已经在本地，真正安装新软件时仍需要下载对应的包文件。

因此安装失败不一定意味着“软件不存在”，也可能是：

- 网络中断。
- DNS 配置错误。
- 软件源地址失效。
- 代理设置错误。
- HTTPS 证书或时间错误。
- 磁盘空间不足。

```
ZL@VM-0-14-ubuntu:~/code/lesson1$ ping www.baidu.com
PING www.a.shifen.com (220.181.111.232) 56(84) bytes of data.
'64 bytes from 220.181.111.232: icmp_seq=1 ttl=251 time=3.37 ms
64 bytes from 220.181.111.232: icmp_seq=2 ttl=251 time=3.39 ms
64 bytes from 220.181.111.232: icmp_seq=3 ttl=251 time=3.37 ms
64 bytes from 220.181.111.232: icmp_seq=4 ttl=251 time=3.36 ms
64 bytes from 220.181.111.232: icmp_seq=5 ttl=251 time=3.37 ms
64 bytes from 220.181.111.232: icmp_seq=6 ttl=251 time=3.40 ms
^C
--- www.a.shifen.com ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5007ms
rtt min/avg/max/mdev = 3.355/3.376/3.402/0.016 ms
ZL@VM-0-14-ubuntu:~/code/lesson1$
```

6.2 ping 能说明什么

6.2.1 ping 可以测试什么

```
ping www.baidu.com
```

`ping` 主要测试：

- 域名能否解析。
- 对端是否能够响应 ICMP 请求。
- 网络延迟和丢包情况。

6.2.2 ping 不能证明软件源可用

能 `ping` 通，不代表能够正常访问某个 HTTPS 软件源。

更接近包管理器实际行为的检查方式是：

```
curl -I https://mirrors.tuna.tsinghua.edu.cn/
wget --spider https://mirrors.tuna.tsinghua.edu.cn/
```

`curl -I` 发起 HTTP/HTTPS 请求并查看响应头；`wget --spider` 只检查目标是否存在，不下载完整文件。

6.3 常见问题排查

6.3.1 apt update 报错

可以依次检查：

```
cat /etc/apt/sources.list
ll /etc/apt/sources.list.d/
curl -I 软件源地址
```

重点确认：

- 软件源链接是否仍然有效。
- 发行版代号是否正确。
- CPU 架构是否包含在该仓库中。
- 第三方源签名密钥是否存在。

6.3.2 yum 找不到包

先确认仓库和缓存：

```
yum repolist
sudo yum makecache
yum search 包名
```

可能原因：

- 仓库已经失效。
- 包名拼写不同。
- 包不在当前发行版源中。

- 包只存在于 EPEL 或第三方仓库。

6.3.3 包管理器被锁住

常见提示会提到 `Could not get lock` 或 `Waiting for cache lock`。

先等待前一个安装结束，再检查是否存在正在运行的包管理进程。强行删除锁文件可能让软件数据库进入不一致状态。

6.3.4 安装成功后却找不到命令

可能原因：

- 包只安装了库，没有安装命令行入口。
- 命令位于未加入 `PATH` 的目录。
- 实际需要安装另一个包。
- 当前 Shell 的命令缓存尚未更新。

可以检查：

```
which 命令名
dpkg -L 包名 | grep bin
rpm -ql 包名 | grep bin
```

6.3.5 磁盘空间不足

检查空间：

```
df -h
du -sh /var/cache/apt
du -sh /var/cache/yum
```

清理缓存前要确认不会影响正在进行的安装任务。

7. 包管理器操作流程与命令速查

7.1 推荐安装流程

7.1.1 操作前

```
确认发行版
→ 确认软件源
→ 更新软件包索引
→ 搜索包名
→ 查看版本、依赖和来源
```

7.1.2 操作中

```
执行安装
→ 阅读依赖和容量提示
→ 确认来源可信
→ 等待安装完成
```

7.1.3 操作后

```
确认命令或服务存在
→ 检查版本
→ 按需清理缓存
```

7.2 apt 速查

```
sudo apt update
apt search 包名
apt show 包名
sudo apt install -y 包名
sudo apt remove 包名
sudo apt purge 包名
sudo apt upgrade
sudo apt clean
sudo apt autoclean
apt policy
apt list --installed | grep 包名
```

7.3 yum 速查

```
sudo yum makecache
yum search 包名
yum info 包名
sudo yum install -y 包名
sudo yum remove -y 包名
sudo yum update
sudo yum clean all
yum repolist
yum list installed | grep 包名
```

7.4 软件源速查

```
cat /etc/apt/sources.list
ll /etc/apt/sources.list.d/
ll /etc/yum.repos.d/
sudo apt update
sudo yum makecache
sudo yum repolist
sudo apt policy
```

7.5 本文核心主线

包管理器并不是简单地“帮我们从网上下载软件”，而是把多个环节串成一条完整链路：

```
软件生态
→ 软件包与软件源
→ 包管理器读取配置
→ 镜像源加速下载
→ 自动解析依赖
→ 安装、升级或卸载
→ 本地软件数据库更新
```

理解这条链路后，遇到安装失败时，就可以分别检查：

1. 包名是否正确。
2. 当前发行版和架构是否匹配。
3. 软件源是否可用。
4. 网络和 DNS 是否正常。
5. 依赖是否能够解析。

6. 包管理器是否持有锁。
7. 磁盘空间是否足够。

本文只整理包管理器相关内容，不展开权限、vim、gcc/g++、make、进度条、Git 或 gdb/cgdb。